Predictive Maintenance System based on Anomaly Detection in Sensor Data from IoT devices

Project Team

Laiba Zulfiqar 21I-1501
Imra Tariq 21I-1521

Session 2021-2025

Supervised by

Dr Muhammad Arshad Islam

Co-Supervised by

Dr Imran Ashraf

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

1 Introduction **1**

1.1 Problem Statement . . . 1

1.1.1 Reason for Developing this System . . . 2

1.1.2 Description . . . 2

1.1.3 Problem Solution . . . 3

1.2 Scope . . . 3

1.3 Modules . . . 4

1.3.1 Module 1: Data Preprocessing . . . 4

1.3.2 Module 2: Model Training . . . 4

1.4 User Classes and Characteristics . . . 4

2 Project Requirements **7**

2.1 Use-case/Event Response Table/Storyboarding . . . 7

2.1.1 Use Case Diagram . . . 7

2.2 Functional Requirements . . . 7

2.2.1 Module 1 . . . 7

2.2.2 Module 2 . . . 8

2.3 Non-Functional Requirements . . . 9

2.3.1 Reliability . . . 9

2.3.1.1 Definition of Failure . . . 9

2.3.1.2 MTBF in Jupyter: . . . 10

2.3.1.3 Consequences of Software Failure: . . . 10

2.3.1.4 Error Detection Strategy . . . 10

2.3.1.5 Error Correction Strategy . . . 10

2.3.1.6 Protection from Failure . . . 11

2.3.2 Usability . . . 11

2.3.3 Performance . . . 12

2.3.3.1 Data Collection Performance . . . 12

2.3.3.2 Data Processing Performance . . . 12

2.3.3.3 Model Training Performance . . . 12

2.3.3.4 User Interface Performance . . . 12

2.3.3.5 Alert Generation Performance . . . 13

2.3.3.6 Data Retrieval Performance 13
2.3.3.7 Backup Performance 13
2.3.4 Security 13
2.4 Domain Model 13

3 System Overview **15**
3.1 Architectural Design 15
3.2 Design Models 15
3.3 Data Design 17

References **19**

List of Figures

1.1 Block diagram of our system. 1

2.1 Use Case Diagram of the system. 8
2.2 Domain Model of our system. 14

3.1 Architectural Design. 15
3.2 Activity diagram of the system. 16
3.3 Class Diagram of the system. 16
3.4 Class-level Sequence Diagram Module 1 17
3.5 Class-level Sequence Diagram Module 2 17

List of Tables

1.1 User Classes and Characteristics 5

Chapter 1

Introduction

In recent years, the industrial sector has witnessed a significant increase in the use of IoT devices. CARE Pvt. Ltd. has its own IoT devices that generate a vast amount of data from multiple sensors (3-Phase Current Sensor, Thermistor, and Vibration Sensor) attached to motors of different equipment. Their devices are working in various industrial areas. They wish to monitor plant operations in real-time and predict potential equipment failures for the following industrial machinery in Pakistan.

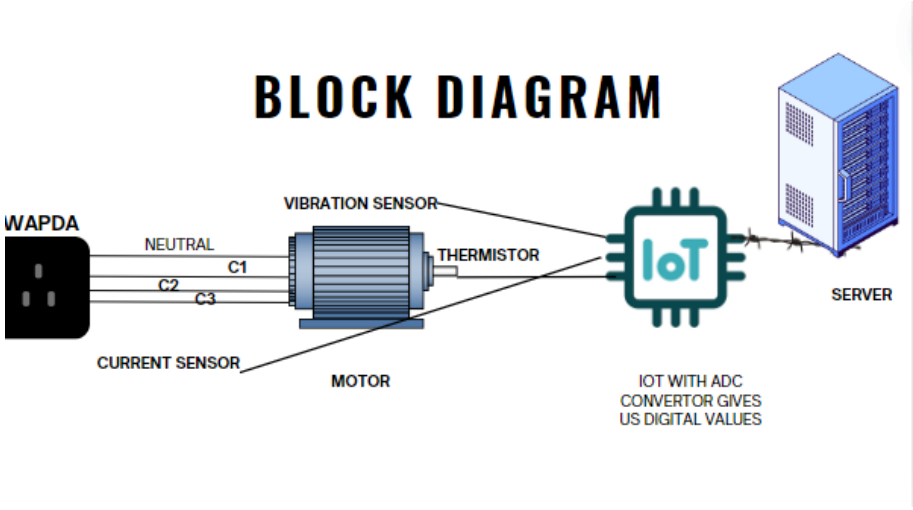

Figure 1.1: Block diagram of our system.

1.1 Problem Statement

This project aims to develop a Predictive Maintenance System that uses anomaly detection in sensor data to identify potential equipment failures.

1.1.1 Reason for Developing this System

The main objective of this project is to create an efficient and functional Predictive Maintenance System for the company, which enables them to carry out their everyday operations and receive alerts before an equipment failure occurs. This solution aims to support the Pakistani industry in sustaining their machinery, extending the lifespan of their equipment, and avoiding catastrophic failures due to machine malfunctions. It aligns with our goal of building sustainable cities by reducing equipment waste through early detection of small faults or anomalies before they escalate into major failures.

1.1.2 Description

We aim to create a Predictive Maintenance System by following these steps:

- Preprocessing the two datasets provided by CARE Pvt. Ltd.:
  - Joyland Rawalpindi (Flying Carpet)
  - Chenab Ltd. Faisalabad (Drill Machine)
- Training our data by analyzing anomalies for prediction within our dataset.
- We are opting for a supervised time series data approach for anomalies in our dataset, as it is labeled.
- LSTM (Long Short-Term Memory) is a more practical and faster approach for large time series datasets compared to ARIMA, especially since CARE's IoT devices fetch data every 5 seconds.
- We will be using a hybrid approach, combining both statistical modeling and machine learning models:
  - Statistical modeling will help set thresholds, standard deviations, and apply mathematical formulas for accurate outputs. However, it may not be recommended for very large datasets due to its computational intensity.
  - Machine learning will help analyze contextual anomalies in our datasets. For example, vibration values in data points that may be normal in some contexts (due to heavy load) but anomalous in specific contexts. We will apply developed machine learning models to test and train our data.
- Predicting equipment failures before they occur.

- After testing our machine learning models on the data, we will create a forecast model based on historical patterns to prevent failures and schedule timely maintenance activities.
- Visualizing our models and Predictive Maintenance System.

1.1.3 Problem Solution

The software being developed will provide a comprehensive Predictive Maintenance System that utilizes IoT sensor data and machine learning techniques to detect anomalies and predict equipment failures before they occur. The system's objectives include:

- Real-time monitoring of sensor data from motors and equipment.
- Analyzing anomalies and providing early alerts for potential failures.
- Extending the life of machinery by optimizing maintenance schedules.
- Reducing downtime caused by unexpected equipment failures.
- Providing a hybrid solution using both statistical modeling and machine learning for anomaly detection.
- Automating the maintenance process by predicting failure patterns and scheduling repairs accordingly.
- Visualizing data and predictive outcomes for better decision-making.

The goal of this project is to create an efficient system that minimizes disruptions in industrial operations and helps companies manage their equipment more effectively.

1.2 Scope

Following are the scope of our project:

- **Data Preprocessing**: The project deals with data from vibration, temperature, and current sensors installed on CARE Pvt Limited's machinery. We will clean our data from noises and handle missing values.
- **Anomaly Detection**: Implemented using both statistical and machine learning techniques to identify unusual patterns in the sensor data. Stats related to machine health will be visualized using a user-friendly interface.

- **Visualization**: Anomalies will be visualized in real-time using Jupyter notebooks, with a green/red light system to indicate the status.
- **Predictive Maintenance**: Predict anomalies and schedule timely maintenance activities based on historical patterns to prevent failures.

1.3 Modules

The following are the modules of the proposed project:

1.3.1 Module 1: Data Preprocessing

This module is responsible for preparing the data before it is fed into the anomaly detection model. The key steps include cleaning the data and handling missing or noisy values.

1. Combine datasets
2. Alter time of all datasets
3. Preprocess datasets
4. Handle Missing values

1.3.2 Module 2: Model Training

Following are the requirements for module 2: Once the data is preprocessed, the next step involves training the anomaly detection model using the cleaned datasets.

1. Load Datasets
2. Apply LSTM
3. Train dataset

1.4 User Classes and Characteristics

Following are the user classes and their characteristics:

User Class	**Description**
Access the collected data	The user accesses sensor data collected by the system.
Logins to the System	The user enters a username and password for authentication.
Selects city and device	The interface provides the user the option to select a city and device and to add another device if needed.
Manages API configuration	Managing APIs related to adding data or devices.
Manages import option	The user imports CSV files into the system.
Selects combine-files option	The system merges multiple files into one single CSV file.
Enters preprocessing data requirements	The system provides the preprocessing steps, allowing the user to choose which steps to apply to their dataset.
Trains model	After preprocessing, the user trains a model to detect anomalies.
Visualizes results	After training the model, the user views the results in the form of graphs.

Table 1.1: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of our project.

2.1 Use-case/Event Response Table/Storyboarding

2.1.1 Use Case Diagram

2.2 Functional Requirements

Following are the Functional Requirements of our project:

2.2.1 Module 1

Module 1 Data Preprocessing This module is responsible for preparing the data before it is fed into the anomaly detection model.The key steps include cleaning the data and handling missing or noisy values.

1. FR1: The system provide an interface to the user
2. FR2: The system shows the login web page for CARE's authorized users.
3. FR3: The system gives access to the authorized users.
4. FR4: The user selects the city and device.
5. FR5: The system provides the user to add another device if they want to.
6. FR6: The user can enter the device's API configuration

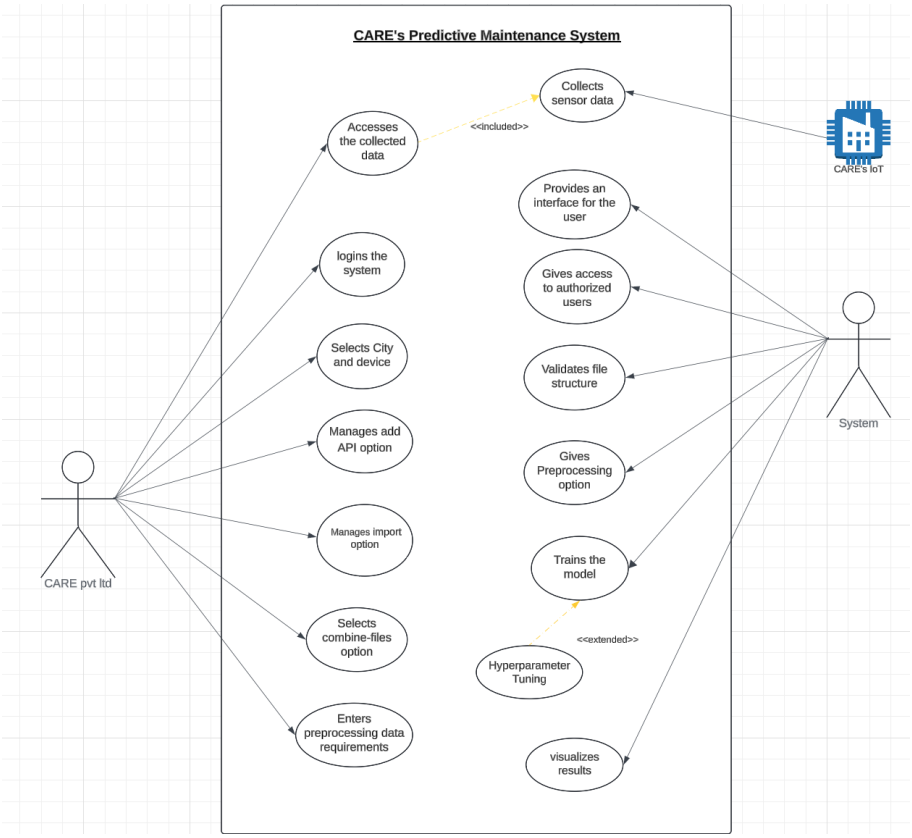

Figure 2.1: Use Case Diagram of the system.

7. FR7: The user can import files in the system

8. FR8: The system checks if the imported files are following the input pattern according to columns otherwise reject the file.

9. FR9:The system provides an option for the user to combine multiple datasets together

10. FR10:The system provides the user preprocessing options, the user can print information,head, descriptive statistics for numerical columns and check for missing values.

2.2.2 Module 2

Following are the requirements for module 2: Once the data is preprocessed, the next step involves training the anomaly detection model using the cleaned datasets.

1. FR1: The system allows the user to select the machine learning algorithm.

2. FR2: The system trains the selected algorithm on the dataset.

3. FR3: The system provides hyperparameter tuning options for the user.
4. FR4: The system allows the user to split the dataset into training and testing sets.
5. FR5: The system generates performance metrics after training.
6. FR6: The system allows users to validate the trained model.
7. FR7: The system supports saving the trained model.
8. FR8: The system visualizes the results.
9. FR9: The system provides an option to retrain the model on new data.
10. FR10: The system supports cross-validation during the training phase.

2.3 Non-Functional Requirements

2.3.1 Reliability

The systems reliability defines how consistently the system handles data preprocessing, data collection, model training, and other tasks without failure.

2.3.1.1 Definition of Failure

Failure to Collect Data: The system does not receive or store data from IoT sensors (vibration, current, or temperature).

Failure to Preprocess Data: Errors in the data preprocessing that prevent the system from preparing data for model training.

Failure to Train or Update the Model: Inability to complete model training or hyperparameter tuning within the predefined time limits.

Failure to Trigger Alerts: The system fails to generate an alert for a detected anomaly or critical maintenance condition.

Failure of the User Interface: The user interface becomes unresponsive or fails to display critical data (e.g., dashboards, charts, device information).

2.3.1.2 MTBF in Jupyter:

The system should run uninterrupted model training for at least 1000 hours before the notebook kernel needs restarting or manual intervention.

2.3.1.3 Consequences of Software Failure:

Loss of Sensor Data: Failure in data collection, the system could lose critical real-time sensor data. This could prevent timely anomaly detection and lead to unscheduled downtime.

Delayed Maintenance: A failure in model training or alert generation could result in missed predictions of equipment failure, leading to increased repair costs and potential machine breakdowns.

User Inconvenience: Failures in the user interface could prevent operators from accessing important system information or performing tasks such as adding devices or managing data.

2.3.1.4 Error Detection Strategy

Data Validation: Incoming data from sensors should undergo integrity checks (e.g., format validation, missing value checks) to detect errors in data collection or transmission.

Monitoring and Alerts: Real-time system monitoring should be in place to detect system-level failures such as service downtime, model training errors, or unresponsive User Interface. Automated alerts should notify system administrators immediately.

2.3.1.5 Error Correction Strategy

Automatic System Recovery: In case of a data collection or preprocessing failure, the system should automatically restart affected services e.g re-initiate data collection.

Fail over Mechanism: If a critical service such as data processing or model training fails, the system should have a secondary service ready to take over, minimizing downtime.

Manual Intervention Alerts: If automated recovery is unsuccessful, the system should alert system administrators for manual intervention.

2.3.1.6 Protection from Failure

Data Storage: As IoT devices collect real-time sensor data, the data should be backed up to CARE's cloud storage to ensure that even during a data collection failure, previous data remains safe and accessible.

Load Balancing: Distributing workload across CARE's multiple servers to ensure the system remains operational even if one server fails.

Data Backup: Regular backups of the data,to recover from failures that result in data loss.

Integration Testing: The system makes sure that data collection, preprocessing, model training, alert generation, and the user interface, work together seamlessly.

2.3.2 Usability

Usability requirements are important for ensuring that the Predictive Maintenance System is user-friendly, efficient, and accessible to all users. These requirements focus on various aspects of usability, including ease of learning, ease of use, error avoidance and recovery, efficiency of interactions, and accessibility. Following are the usability Requirements for CARE's Predictive Maintenance System:

USE-1: The PMS shall allow a user to access the most recent sensor data (vibration, current, temperature) with a single click from the dashboard, minimizing navigation time and improving efficiency.

USE-2: The PMS shall enable users to add new IoT devices to the system through a streamlined process, allowing users to complete the task in three steps or less, including data validation to ensure proper setup.

USE-3: The PMS shall validate user inputs in real time, providing immediate feedback for incorrect entries to prevent errors during data submission.

USE-4: The PMS shall provide a single interface for users to manage alerts, allowing users to set thresholds for multiple sensors simultaneously and providing a summary view of all active alerts.

USE-5: The PMS shall allow a user to access real-time data updates on the dashboard without requiring manual refreshes to support timely decision-making.

USE-6: The PMS shall display sensor data and alerts in visual formats (e.g., graphs, charts) to facilitate quick comprehension of system status and trends, reducing the cognitive load on users when interpreting data

USE-7: Batch Processing for File Imports The PMS shall enable users to import multiple files at once for data analysis, reducing the time spent on manual imports and allowing

for efficient handling of data from various sources.

USE-8: The PMS shall allow users to retrieve historical data for any device with a single query, enabling users to analyze trends and performance without unnecessary delays.

USE-9: The PMS shall provide users with an easy mechanism to acknowledge alerts directly from the notification area, allowing users to quickly confirm they are aware of and addressing issues.

2.3.3 Performance

Performance is crucial for the effectiveness of the CARE Predictive Maintenance System. This section defines specific performance requirements that the system must achieve to ensure efficient data collection, processing, model training, and user interactions. Meeting these goals will enhance system reliability and improve the user experience.

2.3.3.1 Data Collection Performance

PER-1: The system shall process and store data from IoT sensors (vibration, current, temperature) within 5 seconds of data generation.

2.3.3.2 Data Processing Performance

PER-2: The PMS shall complete data preprocessing tasks (including normalization and filtering) for incoming sensor data.

PER-3: The system shall perform real-time anomaly detection on incoming data streams with a processing time of less than 5 seconds per data entry to ensure timely alerts.

2.3.3.3 Model Training Performance

PER-4: The PMS shall train the predictive maintenance model on historical data sets of up to 100,000 entries in less than 30 minutes.

PER-5: The system shall allow hyper-parameter tuning to be completed in no more than 15 minutes for each model iteration, ensuring rapid improvement of model performance.

2.3.3.4 User Interface Performance

PER-6: The user interface shall respond to user inputs (clicks, selections) within 1 second to ensure a smooth and responsive user experience.

PER-7: The PMS shall load dashboard visualizations (charts, graphs) in under 3 seconds after the user requests the data, enhancing usability and efficiency.

2.3.3.5 Alert Generation Performance

PER-8: The system shall generate alerts for detected anomalies within 10 seconds of identification, ensuring users are promptly informed of critical conditions.

2.3.3.6 Data Retrieval Performance

PER-9: The PMS shall retrieve and display historical data (monthly) within 5 seconds upon user request, allowing for quick analysis and decision-making.

2.3.3.7 Backup Performance

PER-10: The system shall perform regular data backups every 24 hours, completing the backup process within 15 minutes to ensure minimal disruption to data accessibility.

2.3.4 Security

SEC-1: The system shall require a username and password for user access to ensure that only authorized personnel can log in.

SEC-2: The system shall restrict user access based on roles (e.g., admin, technician, viewer) to limit permissions to sensitive actions and data.

SEC-:3 The system shall detect unauthorized access attempts within 10 minutes and alert administrators for a quick response.

SEC-:4 The system shall automatically log users out after a period of inactivity (e.g., 15 minutes) to reduce the risk of unauthorized access.

2.4 Domain Model

Domain Model

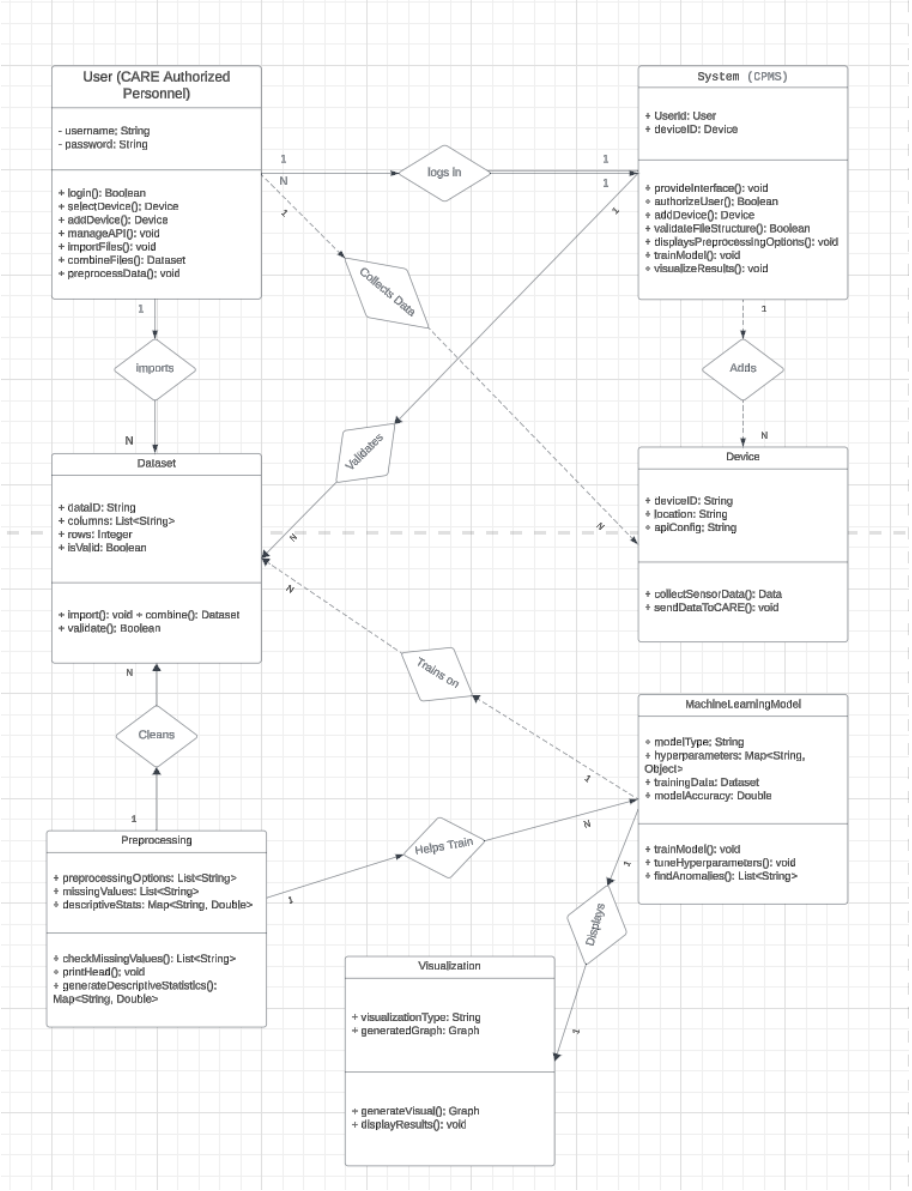

Figure 2.2: Domain Model of our system.

Chapter 3

System Overview

3.1 Architectural Design

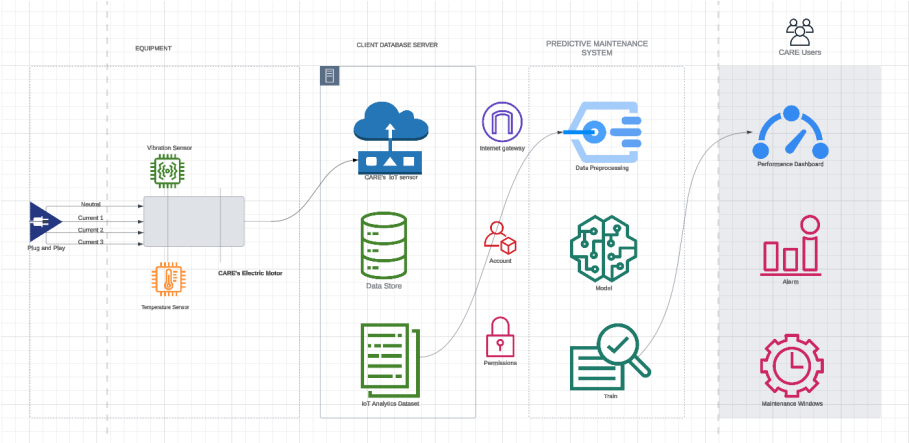

Figure 3.1: Architectural Design.

3.2 Design Models

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

Design Models for Object Oriented Development Approach

The applicable models for the project using object oriented development approach may include:

• Activity Diagram

Figure 3.2: Activity diagram of the system.

• Class Diagram

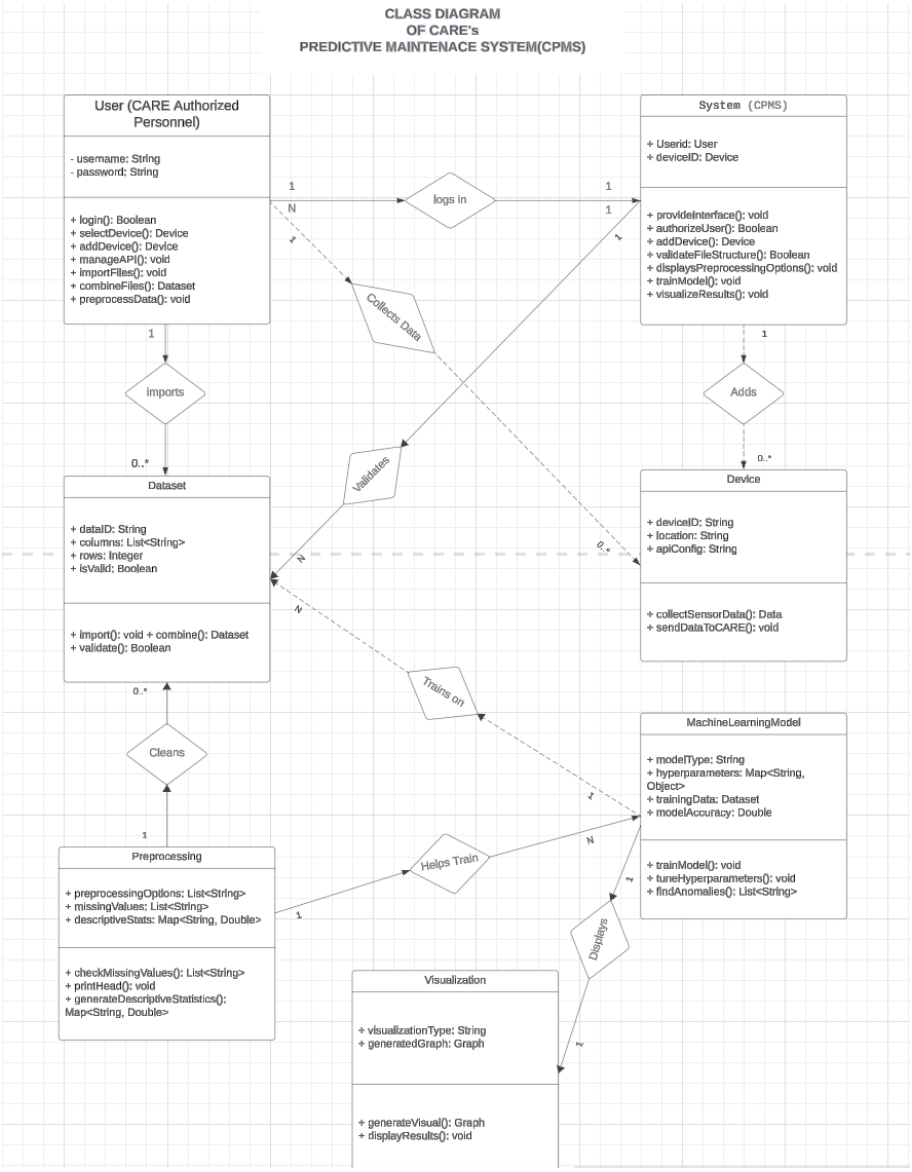

Figure 3.3: Class Diagram of the system.

• Class-level Sequence Diagram

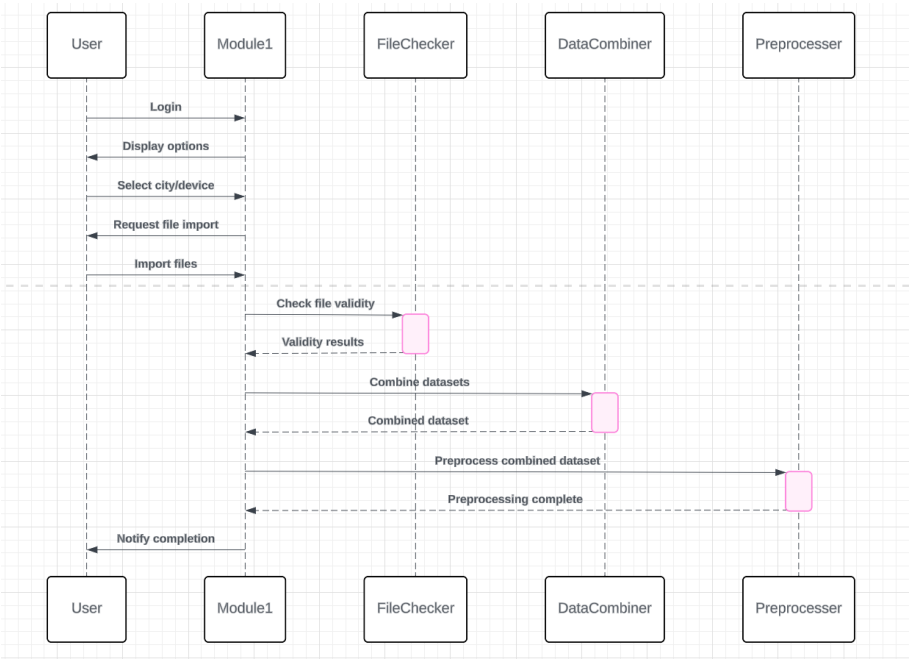

Figure 3.4: Class-level Sequence Diagram Module 1

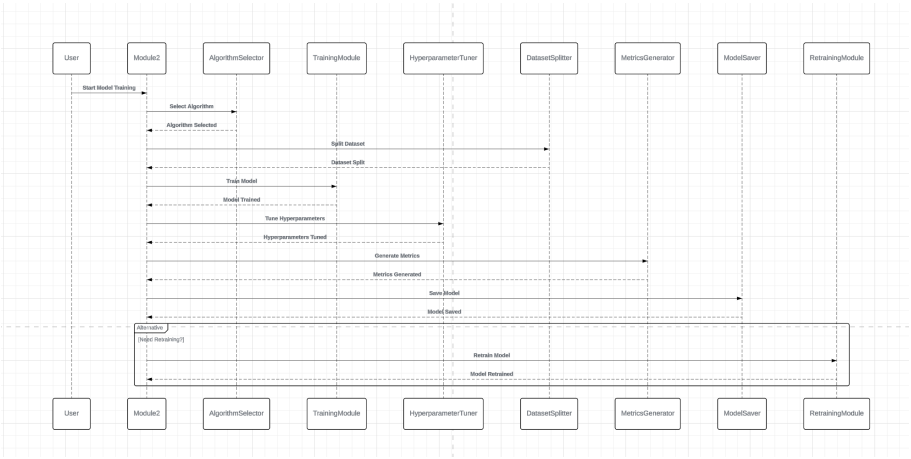

Figure 3.5: Class-level Sequence Diagram Module 2

3.3 Data Design

- CARE's database
- OUR SYSTEMS DATABASE for login

[1, 2, 3, 4, 5]

Bibliography

[1] Xiaoling Chen. Application of iot and ai in building smart cities. In *E3S Web of Conferences*, 2020.

[2] A Kolyshkin and S Nazarovs. Stability of slowly diverging flows in shallow water. *Mathematical Modeling and Analysis*, 2007.

[3] John Smith. *Predictive Maintenance in the Industrial Sector: Trends and Future Prospects*. PhD thesis, University of Technology, 2018.

[4] Li Wang and Xing Yang. Automated predictive maintenance in modern railway infrastructure: A comprehensive review. *Data in Brief*, 2023.

[5] Shunhua Yang, Hui Zhang, and Sungwoo Kim. Safety of hydrogen fuel cell vehicles in fire scenarios: A review. *Journal of Electrical Engineering & Technology*, 13(5):2065–2072, 2018.